A Canonical Quaternionic Intermediate Representation
for Single-Qubit Quantum Circuits

John G. Van Geem
RQM Technologies
orcid: 0009-0002-4003-8452

Date 2026-04-09

Abstract

This paper defines a canonical quaternionic intermediate representation for single-qubit circuit segments. Building on the fixed quaternion–SU(2) correspondence established in *Foundations of Quaternionic Quantum Computing*, it treats the physically relevant action of a single-qubit gate modulo global phase by first choosing an SU(2) representative and then representing that representative by a unit quaternion

$$q = w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k}.$$

The paper introduces a compiler-facing IR object

$$\mathrm{u1q}(w, x, y, z),$$

formalizes its unit-norm and sign-ambiguity constraints, and defines deterministic normalization and canonicalization rules. The canonical representative is chosen from $\{q, -q\}$ by a closed-hemisphere convention $w > 0$, together with an explicit equatorial tie-break when $w = 0$. This yields a shortest-geodesic, deterministic internal form suitable for fusion and rewrite pipelines. Explicit translations are given for the identity, Pauli gates, H, S, T and their adjoints, and axis rotations $R_x(\theta), R_y(\theta), R_z(\theta)$. Composition of adjacent single-qubit gates is shown to correspond to quaternion multiplication followed by re-canonicalization, and worked examples illustrate the relation between named gate syntax, canonical IR, and single-qubit segments extracted from larger circuits. The scope is limited to single-qubit gate action modulo global phase; no multi-qubit IR, optimization guarantee, or hardware claim is made.

Keywords: quaternionic intermediate representation; single-qubit circuits; compiler IR; SU(2); unit quaternions; circuit canonicalization

1 Introduction

The first paper in this series, *Foundations of Quaternionic Quantum Computing*, established a conservative mathematical point: after removal of overall phase, a single-qubit gate admits a representative in SU(2), and under a fixed standard convention that representative is modeled exactly by a unit quaternion. The present paper moves that statement from mathematical language into software architecture language. Its purpose is to specify a deterministic internal representation for *single-qubit circuit structure*.

Compiler pipelines need internal forms that are more rigid than frontend surface syntax. Real circuit languages expose named gates, parameters, control flow, and scheduling constructs [1, 2]. That

is appropriate at the program boundary, but it does not by itself supply a canonical representation of single-qubit action. Distinct textual gate lists may denote the same physical action modulo global phase. Likewise, two distinct SU(2) representatives may differ only by sign. A simplifier or merger that operates only on gate names therefore has to recover equivalence information that is not explicit in the syntax.

The claim of this paper is modest. It does not propose a new quantum language, and it does not deny the standard matrix formulation of single-qubit gates. Instead, it defines a canonical intermediate representation layer for single-qubit gate segments. The representation is quaternionic because the quaternion model makes composition and sign ambiguity explicit while remaining exactly equivalent to the standard SU(2) representative after global phase has been removed.

The resulting object is called u1q. It is a quadruple

$$(w, x, y, z) \in \mathbb{R}^4, \qquad w^2 + x^2 + y^2 + z^2 = 1,$$

interpreted as the unit quaternion $q = w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k}$. Canonicalization is then a separate deterministic step: normalize, resolve the sign ambiguity $q \sim -q$, and choose a unique representative on a closed hemisphere of S^3. This produces an internal form that is suitable for gate fusion, equality checks, and segment-level rewrites.

The scope is limited to single-qubit circuits only. Entangling gates appear only as boundaries that terminate single-qubit segments. No claim is made about multi-qubit quaternionic ontologies, backend execution, or performance advantage.

2 Background and dependency on Foundations of Quaternionic Quantum Computing

2.1 Gates, phase, and the fixed convention

We carry forward without modification the convention fixed in *Foundations of Quaternionic Quantum Computing*. Single-qubit gates live fundamentally in U(2). Physical pure-state action is unchanged if a gate is multiplied by an overall phase. The IR defined here therefore models single-qubit gate action *modulo global phase*.

Proposition 2.1 (Single-qubit gates admit SU(2) representatives)**.** *For every $U \in \mathrm{U}(2)$ there exist $\chi \in \mathbb{R}$ and $V \in \mathrm{SU}(2)$ such that*

$$U = e^{i\chi} V.$$

The representative V is unique up to multiplication by -1.

Proof. This is the same determinant-removal argument used in *Foundations of Quaternionic Quantum Computing*. Choose χ such that $e^{2i\chi} = \det U$. Then $V = e^{-i\chi} U$ has determinant 1, so $V \in \mathrm{SU}(2)$. Any second determinant-one representative differs from V by a phase whose square is 1, hence by ± 1. □

The quaternion convention is also inherited from *Foundations of Quaternionic Quantum Computing*:

$$\Phi(a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}) = \begin{pmatrix} a - di & -c - bi \\ c - bi & a + di \end{pmatrix} = aI - i(b\sigma_x + c\sigma_y + d\sigma_z). \tag{1}$$

Under this convention the quaternion basis units align directly with the Pauli axes:

$$\Phi(\mathbf{i}) = -i\sigma_x, \qquad \Phi(\mathbf{j}) = -i\sigma_y, \qquad \Phi(\mathbf{k}) = -i\sigma_z.$$

Proposition 2.2 (Unit quaternions model SU(2))**.** *The restriction of* Φ *to the unit quaternions*

$$\mathrm{Sp}(1) = \{q \in \mathbb{H} : |q| = 1\}$$

is a Lie-group isomorphism from Sp(1) *onto* SU(2)*.*

Proof. This is the standard result established in *Foundations of Quaternionic Quantum Computing*. The key points are that Φ preserves the quaternion multiplication rules and that $\det \Phi(q) = |q|^2$. □

Corollary 2.3 (The only ambiguity is $q \sim -q$)**.** *If* $q, q' \in \mathrm{Sp}(1)$ *represent the same single-qubit gate action modulo global phase under the convention* (1)*, then* $q' = q$ *or* $q' = -q$*.*

Proof. By the previous proposition, unit quaternions correspond bijectively to SU(2) elements. By the first proposition, the SU(2) representative of a single-qubit gate action is unique up to sign. □

2.2 Why this matters for compilers

From a compiler perspective, the point of the previous subsection is simple. Named gate syntax and canonical IR are different layers.

- A frontend may say `H ; T ; Rz(pi/3)` or some equivalent language surface.
- The compiler may first translate each gate into a unit quaternion.
- The compiler may then compose adjacent single-qubit gates by quaternion multiplication.
- The compiler may finally store only the resulting canonical $\mathrm{u1q}(w, x, y, z)$ tuple.

This is not a change of mathematics. It is a change of internal representation.

3 Definition of the canonical quaternionic IR

Definition 3.1 (Global-phase equivalence on U(2))**.** *For single-qubit gates* $U, V \in \mathrm{U}(2)$*, write*

$$U \sim_{\mathrm{gp}} V$$

if there exists $\chi \in \mathbb{R}$ *such that* $U = e^{i\chi} V$*.*

Definition 3.2 (Quaternionic single-qubit IR element)**.** *A* quaternionic single-qubit IR element*, written*

$$\mathrm{u1q}(w, x, y, z),$$

is a quadruple of real numbers satisfying

$$w^2 + x^2 + y^2 + z^2 = 1.$$

It is interpreted as the unit quaternion

$$q = w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k} \in \mathrm{Sp}(1),$$

and therefore, under (1)*, as an* SU(2) *representative of a single-qubit gate action modulo global phase.*

Remark 3.3. The IR stores a representative of the physical single-qubit action, not the full U(2) gate with its arbitrary global phase. This is why the natural ambiguity is the sign pair $\{q, -q\}$ rather than an arbitrary complex phase.

Definition 3.4 (Equivalence relation on unit quaternions)**.** *For unit quaternions $q, q' \in \mathrm{Sp}(1)$, write*

$$q \sim_{\pm} q'$$

if $q' = q$ or $q' = -q$.

Definition 3.5 (Single-qubit segment)**.** *A* single-qubit segment *on wire a is a maximal consecutive run of gates that act only on wire a, bounded on either side by the beginning or end of the circuit, or by an operation that is outside the single-qubit unitary model for that wire. In particular, an entangling gate touching wire a terminates the segment.*

4 Canonicalization rules

4.1 Normalization

Exact mathematics lives on the unit sphere $\mathrm{Sp}(1) \subset \mathbb{R}^4$, but software implementations often carry floating-point drift.

Definition 4.1 (Normalization map)**.** *For any nonzero quadruple*

$$\tilde{q} = (\tilde{w}, \tilde{x}, \tilde{y}, \tilde{z}) \in \mathbb{R}^4 \setminus \{0\},$$

define

$$\mathrm{Norm}(\tilde{q}) = \frac{\tilde{q}}{\|\tilde{q}\|}, \qquad \|\tilde{q}\| = \sqrt{\tilde{w}^2 + \tilde{x}^2 + \tilde{y}^2 + \tilde{z}^2}.$$

Remark 4.2. The zero quadruple does not represent any gate action and is therefore outside the domain of canonicalization.

4.2 Sign canonicalization

The quotient ambiguity $q \sim_{\pm} -q$ must be resolved deterministically.

Definition 4.3 (Canonical hemisphere)**.** *Let $\mathcal{H}_{\mathrm{can}} \subset \mathrm{Sp}(1)$ be the set of unit quaternions satisfying either*

(a) $w > 0$, or

(b) $w = 0$ and the first nonzero component among (x, y, z) is positive.

Definition 4.4 (Sign-canonicalization map)**.** *For $q = (w, x, y, z) \in \mathrm{Sp}(1)$, define* $\mathrm{SignCan}(q)$ *by*

$$\mathrm{SignCan}(q) = \begin{cases} q, & w > 0, \\ -q, & w < 0, \\ q, & w = 0 \text{ and the first nonzero entry of } (x, y, z) \text{ is positive}, \\ -q, & w = 0 \text{ and the first nonzero entry of } (x, y, z) \text{ is negative}. \end{cases}$$

Definition 4.5 (Canonical representative)**.** *For any nonzero quaternionic quadruple $\tilde{q} \in \mathbb{R}^4 \setminus \{0\}$, define its canonical representative by*

$$\mathrm{Can}(\tilde{q}) = \mathrm{SignCan}(\mathrm{Norm}(\tilde{q})).$$

If $q \in \mathrm{Sp}(1)$ is already unit norm, then $\mathrm{Can}(q) = \mathrm{SignCan}(q)$.

Proposition 4.6 (Canonicalization selects one representative from each sign class)**.** *Every equivalence class $\{q, -q\}$ with $q \in \mathrm{Sp}(1)$ contains exactly one element of $\mathcal{H}_{\mathrm{can}}$.*

Proof. If $w \neq 0$, then exactly one of q or $-q$ has positive real part. If $w = 0$, then q and $-q$ have opposite signs in the first nonzero component of (x, y, z), so exactly one satisfies the equatorial tie-break rule. Hence each class intersects $\mathcal{H}_{\mathrm{can}}$ in exactly one point. □

Remark 4.7 (Shortest-geodesic convention)*.* Write a unit quaternion in axis–angle form as

$$q = \cos(\theta/2) + \sin(\theta/2)(n_x\mathbf{i} + n_y\mathbf{j} + n_z\mathbf{k}).$$

The condition $w = \cos(\theta/2) \geq 0$ implies $\theta \in [0, \pi]$. The canonical hemisphere therefore chooses the representative whose rotation angle lies on the shorter closed interval. When $w = 0$, corresponding to $\theta = \pi$, both signs have the same geodesic length, so the equatorial tie-break supplies determinism rather than new physics.

Remark 4.8 (Exact versus numerical equality)*.* The mathematical equivalence relation is exact. In floating-point implementations, comparisons should be performed with explicit tolerances after normalization and canonicalization. That is an implementation detail, not part of the exact equivalence relation itself.

5 Translation from named gates into quaternion IR

Named gate syntax is not itself canonical. Different frontends may expose the same physical action with different symbols, parameters, or decompositions. The translation layer maps those named gates into a common quaternionic form.

Proposition 5.1 (Standard named gates have explicit canonical u1q entries)**.** *Under the convention* (1)*, the following single-qubit gates map to the indicated canonical unit quaternions modulo global phase:*

Gate	*Chosen representative modulo global phase*	*Canonical quaternion*
I	I	$(1, 0, 0, 0)$
X	$-iX$	$(0, 1, 0, 0)$
Y	$-iY$	$(0, 0, 1, 0)$
Z	$-iZ$	$(0, 0, 0, 1)$
H	$-iH$	$(0, \frac{1}{\sqrt{2}}, 0, \frac{1}{\sqrt{2}})$
S	*determinant-one representative of* S	$(\frac{1}{\sqrt{2}}, 0, 0, \frac{1}{\sqrt{2}})$
$S^\dagger$	*determinant-one representative of* $S^\dagger$	$(\frac{1}{\sqrt{2}}, 0, 0, -\frac{1}{\sqrt{2}})$
T	*determinant-one representative of* T	$(\cos\frac{\pi}{8}, 0, 0, \sin\frac{\pi}{8})$
$T^\dagger$	*determinant-one representative of* $T^\dagger$	$(\cos\frac{\pi}{8}, 0, 0, -\sin\frac{\pi}{8})$
$R_x(\theta)$	$R_x(\theta)$	$(\cos\frac{\theta}{2}, \sin\frac{\theta}{2}, 0, 0)$
$R_y(\theta)$	$R_y(\theta)$	$(\cos\frac{\theta}{2}, 0, \sin\frac{\theta}{2}, 0)$
$R_z(\theta)$	$R_z(\theta)$	$(\cos\frac{\theta}{2}, 0, 0, \sin\frac{\theta}{2})$

Proof. The rotation families follow immediately from the quaternion axis–angle form

$$q = \cos(\theta/2) + \sin(\theta/2)(n_x\mathbf{i} + n_y\mathbf{j} + n_z\mathbf{k}).$$

The Pauli gates are the π-rotations about the coordinate axes, so their canonical representatives are $\mathbf{i}, \mathbf{j}, \mathbf{k}$. The S and T families are the z-axis quarter- and eighth-turns after removal of determinant phase. For the Hadamard gate,

$$H = \frac{\sigma_x + \sigma_z}{\sqrt{2}},$$

so after removing a phase factor one obtains the π-rotation about the axis $(1, 0, 1)/\sqrt{2}$, giving $q_H = (0, 1/\sqrt{2}, 0, 1/\sqrt{2})$. □

Remark 5.2. The table above concerns semantic gate action modulo global phase. It is therefore a representation table, not a commitment to any specific frontend syntax spelling. For example, `sdg`, $S^\dagger$, or an equivalent phase-gate spelling may all map to the same canonical tuple.

6 Composition and segment representation

The group law on unit quaternions makes segment fusion immediate.

Proposition 6.1 (Composition of adjacent single-qubit gates)**.** *Let $g_1, \ldots, g_m$ be consecutive single-qubit gates acting on the same wire, listed in application order. Let $q_1, \ldots, q_m$ be their unit-quaternion representatives under* (1)*. Then the fused single-qubit action of the segment is represented by*

$$q_{\text{seg}} = \text{Can}(q_m q_{m-1} \cdots q_1).$$

Proof. By the convention carried forward from *Foundations of Quaternionic Quantum Computing*,

$$\Phi(q_m q_{m-1} \cdots q_1) = \Phi(q_m)\Phi(q_{m-1}) \cdots \Phi(q_1),$$

which is exactly the SU(2) representative of sequential application of the segment gates. Canonicalization then chooses the deterministic representative of the resulting sign class. □

Corollary 6.2 (Canonicalization may be applied after each fusion step)**.** *In exact arithmetic, repeated canonicalization during a left-to-right or right-to-left fold produces the same physical single-qubit action as canonicalizing once at the end. In floating-point arithmetic, intermediate re-normalization may improve stability without changing the intended exact semantics.*

Remark 6.3. This is the principal compiler-facing advantage of the representation. A frontend may emit a list of named gates, but an optimizer may replace the list by a single canonical u1q payload whenever a maximal single-qubit segment is identified.

7 Worked examples

The examples below use a semicolon-separated gate-list style close to common quantum assembly notations [1, 2]. The point is not to fix a syntax standard, but to show the translation boundary between named gates and canonical IR.

Example 7.1 (A mixed named and parametric segment). Consider the single-qubit segment

$$H \ ; \ T \ ; \ R_z(\pi/3).$$

The gatewise quaternion entries are

$$q_H = \left(0, \frac{1}{\sqrt{2}}, 0, \frac{1}{\sqrt{2}}\right), \qquad q_T = \left(\cos\frac{\pi}{8}, 0, 0, \sin\frac{\pi}{8}\right),$$

$$q_{R_z(\pi/3)} = \left(\frac{\sqrt{3}}{2}, 0, 0, \frac{1}{2}\right).$$

Composing in application order gives

$$q_{\text{seg}} = \mathrm{Can}\big(q_{R_z(\pi/3)} q_T q_H\big) \approx (0.5609855,\ -0.4304593,\ -0.5609855,\ -0.4304593).$$

The named syntax and the canonical tuple are distinct objects: the former is a frontend presentation, while the latter is a deterministic segment payload.

Example 7.2 (Fusion of two axis rotations). For

$$R_x(\pi/2) \ ; \ R_y(\pi/2),$$

the quaternions are

$$q_x = \left(\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2}, 0, 0\right), \qquad q_y = \left(\frac{\sqrt{2}}{2}, 0, \frac{\sqrt{2}}{2}, 0\right).$$

Hence

$$\mathrm{Can}(q_y q_x) = \left(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, -\frac{1}{2}\right).$$

This exact tuple is often easier to compare and store than the original rotation syntax.

Example 7.3 (Named syntax can collapse to a simpler canonical payload). Consider

$$Z \ ; \ S \ ; \ T.$$

The gatewise representatives are

$$q_Z = (0, 0, 0, 1), \qquad q_S = \left(\frac{1}{\sqrt{2}}, 0, 0, \frac{1}{\sqrt{2}}\right), \qquad q_T = \left(\cos\frac{\pi}{8}, 0, 0, \sin\frac{\pi}{8}\right).$$

The fused canonical result is

$$\mathrm{Can}(q_T q_S q_Z) = \left(\cos\frac{\pi}{8}, 0, 0, -\sin\frac{\pi}{8}\right),$$

which is the canonical representative of the same single-qubit action modulo global phase. The example illustrates why a canonical IR is more informative for simplification than the original named gate list.

Example 7.4 (Segment extraction inside a larger circuit). Consider the two-wire circuit

$$\text{wire 0: } H \ ; \ \mathrm{CNOT}_{0,1} \ ; \ T^\dagger \ ; \ R_x(\pi/4),$$

$$\text{wire 1: } S \ ; \ \mathrm{CNOT}_{0,1} \ ; \ H.$$

The entangling gate is outside the present IR. Its only role here is to split single-qubit segments.

On wire 0, the segment before the entangling boundary is the single gate H, while the segment after the boundary is

$$T^\dagger \; ; \; R_x(\pi/4),$$

with fused canonical payload

$$\mathrm{Can}\big(q_{R_x(\pi/4)} q_{T^\dagger}\big) \approx (0.8535534,\, 0.3535534,\, 0.1464466,\, -0.3535534).$$

On wire 1, the entangling gate similarly separates the one-gate segments S and H. The example is intentionally modest: the paper does not define the internal representation of the entangling gate, only the single-qubit segments around it.

8 Standard mathematics vs. RQM Technologies framing

8.1 Standard facts used here

The following ingredients are standard and not specific to this series:

(1) single-qubit gates live in U(2);

(2) after removing global phase, they admit representatives in SU(2);

(3) the unit quaternions form a Lie group isomorphic to SU(2);

(4) the only residual ambiguity in that model is $q \sim -q$; and

(5) sequential single-qubit gate composition corresponds to quaternion multiplication under the fixed convention (1).

8.2 Project-specific design choices

The project-specific content of this paper is representational rather than physical:

(a) the IR object name u1q;

(b) the decision to store single-qubit action as a four-real quaternion tuple;

(c) the closed-hemisphere sign convention $w > 0$ plus the equatorial tie-break rule; and

(d) the interpretation of maximal single-qubit runs between entangling boundaries as the natural unit of fusion for this layer.

These are software-architecture choices. They are intended to make equality, fusion, and simplification deterministic. They are not claims about new physics, and they do not imply that named gate syntax or matrix methods are incorrect.

9 Scope, limitations, and non-claims

This paper makes several non-claims explicit:

(1) It does *not* define a full multi-qubit quaternionic circuit IR.

(2) It does *not* specify the internal semantics of entangling gates.

(3) It does *not* prove a runtime, memory, or optimization advantage.

(4) It does *not* replace existing SDKs, assembly languages, or compiler frameworks.

(5) It does *not* define a backend execution format.

(6) It does *not* claim that standard matrix methods are wrong or insufficient.

The paper defines only a canonical representation layer for single-qubit gate action modulo global phase.

10 Conclusion

The main result of this paper is representational. A single-qubit gate action modulo global phase can be stored deterministically as a canonical unit quaternion

$$\mathrm{u1q}(w, x, y, z),$$

with explicit normalization and sign-resolution rules. The representation is equivalent to the standard SU(2) representative after global phase removal, inherits a simple composition law from quaternion multiplication, and provides a clean internal target for single-qubit segment fusion.

In that sense, the paper is a bridge from quaternionic single-qubit geometry to compiler IR design. It does not solve all compiler problems, but it does make one layer precise: the canonical internal representation of single-qubit gate segments.

A Lookup table for common gates

Gate	Canonical quaternion	Notes
I	$(1, 0, 0, 0)$	identity
X	$(0, 1, 0, 0)$	π-rotation about x
Y	$(0, 0, 1, 0)$	π-rotation about y
Z	$(0, 0, 0, 1)$	π-rotation about z
H	$(0, \frac{1}{\sqrt{2}}, 0, \frac{1}{\sqrt{2}})$	π-rotation about $(1, 0, 1)/\sqrt{2}$
S	$(\frac{1}{\sqrt{2}}, 0, 0, \frac{1}{\sqrt{2}})$	quarter-turn about z
T	$(\cos\frac{\pi}{8}, 0, 0, \sin\frac{\pi}{8})$	eighth-turn about z
$R_x(\theta)$	$(\cos\frac{\theta}{2}, \sin\frac{\theta}{2}, 0, 0)$	exact family
$R_y(\theta)$	$(\cos\frac{\theta}{2}, 0, \sin\frac{\theta}{2}, 0)$	exact family
$R_z(\theta)$	$(\cos\frac{\theta}{2}, 0, 0, \sin\frac{\theta}{2})$	exact family

B Implementation notes

The mathematical definition of Can is exact, but an implementation should usually include three practical checks:

(a) reject or special-case near-zero input before normalization;

(b) re-normalize after quaternion multiplication to control floating-point drift; and

(c) evaluate the $w = 0$ tie-break with a tolerance so that numerically small values are not misclassified.

These are engineering details layered on top of the exact canonicalization rule.

C Companion figures

Publication-ready SVG figures for the package are provided in `artifacts/figures/`. They illustrate the frontend-to-IR translation pipeline, the quotient identification $q \sim -q$, the canonical hemisphere selection rule, and segment extraction around entangling boundaries.

References

[1] Andrew W. Cross, Ali Javadi-Abhari, Thomas Alexander, Niel de Beaudrap, Lev S. Bishop, Steven Heidel, Colm A. Ryan, Prasahnt Sivarajah, John A. Smolin, Jay M. Gambetta, and Blake R. Johnson. Openqasm 3: A broader and deeper quantum assembly language. *ACM Transactions on Quantum Computing*, 3(3), 2022. doi: 10.1145/3505636.

[2] Robert S. Smith, Michael J. Curtis, and William J. Zeng. A practical quantum instruction set architecture. *arXiv preprint arXiv:1608.03355*, 2016.